SQL / TASK 1

Basic Data Retrieval & Filtering

Complete solution | MySQL syntax | employees table

Purpose. Practise selecting columns, filtering rows, sorting results, limiting output, and validating query results using an **employees** table.

Assumption. Examples use MySQL syntax with columns such as first_name, last_name, department, salary, and hire_date.

1 Column Selection

Select only the columns you need instead of pulling every column with SELECT *.

```
SELECT first_name, last_name, salary
FROM employees;
```

Calculated column: 10% salary increase

```
SELECT first_name, salary,
       salary * 1.1 AS new_salary
FROM employees;
```

The alias new_salary gives the calculated result a clear column name.

2 Filtering with WHERE

Comparison operators

```
-- Salary equal to 50000
SELECT * FROM employees WHERE salary = 50000;
-- Salary not equal to 50000
SELECT * FROM employees WHERE salary != 50000;
-- Salary greater than 50000
SELECT * FROM employees WHERE salary > 50000;
-- Salary less than 50000
SELECT * FROM employees WHERE salary < 50000;
-- Salary greater than or equal to 50000
SELECT * FROM employees WHERE salary >= 50000;
-- Salary less than or equal to 50000
SELECT * FROM employees WHERE salary <= 50000;
```

IMPORTANT salary > 50000 excludes 50000, while salary >= 50000 includes it.

BETWEEN: select a range (inclusive)

```sql
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 60000;

-- Employees hired during calendar year 2023
SELECT *
FROM employees
WHERE hire_date BETWEEN '2023-01-01' AND '2023-12-31';
```

BETWEEN includes both endpoints. For date-time columns that may contain times, a half-open range (on or after the start, before the next year) is often safer.

LIKE and NOT LIKE: pattern matching

```sql
-- Last names starting with S
SELECT * FROM employees WHERE last_name LIKE 'S%';
-- Last names not starting with S
SELECT * FROM employees WHERE last_name NOT LIKE 'S%';
-- First names ending with a
SELECT * FROM employees WHERE first_name LIKE '%a';
-- First names with 'a' as the second character
SELECT * FROM employees WHERE first_name LIKE '_a%';
```

TIP **%** matches zero or more characters. _ matches exactly one character. Case sensitivity depends on the database collation.

Combine conditions with AND / OR

```sql
-- Both conditions must be true
SELECT *
FROM employees
WHERE department = 'Sales'
  AND salary > 50000;

-- Either department can match
SELECT *
FROM employees
WHERE department = 'Sales'
   OR department = 'Marketing';

-- Use parentheses to make mixed conditions clear
SELECT *
FROM employees
WHERE department = 'Sales'
  AND (salary > 50000 OR salary = 45000);
```

Handle NULL values

```sql
-- Employees whose salary is missing
SELECT * FROM employees WHERE salary IS NULL;
-- Employees whose salary is present
SELECT * FROM employees WHERE salary IS NOT NULL;
```

REMEMBER Always use IS NULL or IS NOT NULL, never = NULL.

3 Sorting with ORDER BY

```sql
-- Lowest salary first
SELECT first_name, salary
FROM employees
ORDER BY salary ASC;

-- Highest salary first
SELECT first_name, salary
FROM employees
ORDER BY salary DESC;

-- Department A-Z, then salary highest to lowest
SELECT first_name, department, salary
FROM employees
ORDER BY department ASC, salary DESC;
```

ASC means ascending and DESC means descending. If order matters, always specify ORDER BY explicitly.

4 Limit and Pagination

```sql
-- First 10 rows (MySQL / PostgreSQL)
SELECT *
FROM employees
LIMIT 10;

-- Skip 10 rows and return the next 10
SELECT *
FROM employees
ORDER BY employee_id
LIMIT 10 OFFSET 10;

-- SQL Server equivalent for the first 10 rows
SELECT TOP 10 *
FROM employees
ORDER BY employee_id;
```

NOTE Use an ORDER BY column for stable pagination. Without ordering, the database does not guarantee which rows appear first.

5 Real-World Business Query

Question: Show Marketing employees hired in 2023 with salaries greater than 50,000.

```sql
SELECT first_name, hire_date, salary
FROM employees
WHERE department = 'Marketing'
  AND hire_date >= '2023-01-01'
  AND hire_date < '2024-01-01'
  AND salary > 50000;
```

The query returns only the requested columns and filters by department, hire date, and salary. The date range covers all of 2023, including date-time values.

6 Main Validation Query

This is the validation query specified in the task description.

```sql
SELECT
    COUNT(*) AS total_filtered,
    MIN(salary) AS min_salary,
    MAX(hire_date) AS latest_hire
FROM employees
WHERE department = 'Sales'
  AND hire_date BETWEEN '1990-01-01' AND '1999-12-31'
  AND last_name LIKE 'S%';
```

COUNT(*) counts matching records. **MIN(salary)** returns the lowest non-NULL salary among them. **MAX(hire_date)** returns the latest matching hire date.

Metric	Example expected result	Status
Row count (total_filtered)	142	Verify against actual database
Minimum salary (min_salary)	42,000	Verify against actual database
Latest hire (latest_hire)	1999-12-24	Verify against actual database

> **NOTE** The three values above are examples supplied in the task brief, not verified results from a real dataset. Actual results depend on the employees table you use.

7 Additional Data Validation Queries

Count all employees

```sql
SELECT COUNT(*) AS total_employees
FROM employees;
```

Count employees with missing salaries

```sql
SELECT COUNT(*) AS missing_salaries
FROM employees
WHERE salary IS NULL;
```

Check salary range and average

```sql
SELECT
    MIN(salary) AS minimum_salary,
    MAX(salary) AS maximum_salary,
    AVG(salary) AS average_salary
FROM employees;
```

For validation, compare query output with the source table, check a few sample rows, and confirm column data types and date formats.

8 Common Errors to Avoid

- Missing commas between selected columns.
- Wrong quote marks, or forgetting quotes around text values.
- Using = NULL instead of IS NULL.
- Mixing AND and OR without parentheses.
- Using an ambiguous date format; prefer YYYY-MM-DD.
- Assuming LIKE is case-sensitive in every database.
- Expecting the sample validation values to match a different dataset.

9 Conclusion

This task practises foundational SQL operations: SELECT, calculated columns, WHERE filters, comparison operators, BETWEEN, LIKE, AND/OR, NULL checks, ORDER BY, LIMIT/OFFSET, and validation with COUNT, MIN, MAX, and AVG. These skills support reliable data retrieval and basic data profiling.